pfi

Python for Fixed Income

COURSE 5 · ADVANCED MACHINE LEARNING

XGBoost, Regularization, and Early Stopping

WEEK 2 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Gradient boosting, with brakes

Day 7's loop

1. Start from the average
2. Fit a small tree to what is left
3. Add a fraction of it
4. Repeat

XGBoost's brakes

`reg_lambda`	shrinks every leaf
`gamma`	minimum gain to split
`reg_alpha`	pulls G toward zero
`subsample`	a share of rows, columns
`early stopping`	stop on unseen rows

The validation rows report and never choose: early stopping stops on rows carved from the training rows.

How XGBoost works

1. Gradient, hessian

g = p - y and h = p(1 - p) for every row

2. Grow a tree

each split has the largest gain in G and H

3. Penalised leaf

-learning_rate x G / (H + lambda)

4. Add and repeat

gamma refuses splits that gain too little

You control

Trees, learning rate, depth, lambda, alpha, gamma, and the sampling.

What comes out

An XGBClassifier: predict_proba is the model probability of default in the file.

One leaf, by hand

```
leaf value = -learning_rate * G / (H + lambda)      g = p - y,  h = p * (1 - p)
```

0.2212

starting probability

-0.097755

left leaf (PAY_0 < 2)

0.833449

right leaf (PAY_0 >= 2)

reg_lambda	Split	Left leaf	Right leaf
0	PAY_0 < 2	-0.0978	0.8360
1	PAY_0 < 2	-0.0978	0.8334
100	PAY_0 < 1	-0.1428	0.4351

By hand: -0.097755 and 0.833449. Equal to XGBoost's to six decimals: XGBoost computes in float32.

gamma 2809 keeps the split; gamma 2810 removes it. A split stays while its gain is above gamma.

n_jobs=1, written out

```
XGBClassifier(n_estimators=300, random_state=SEED, n_jobs=1)
```

Why it is written out

XGBoost's default uses every core. With row or column sampling its results can depend on the thread count: the same code gives other probabilities on another machine.

Setting used	Value
eta	0.3
max_depth	6
lambda	1
alpha	0
gamma	0
subsample	1

The constructor shows None for most settings; the fitted model's saved config shows the values used.

Where the name comes from

Tianqi Chen and Carlos Guestrin, August 2016

"XGBoost: A Scalable Tree Boosting System"

Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pages 785 to 794

The library's name, and "tree boosting": what day 7 built by hand.

Source, read 5 October 2026: the Crossref record of the paper (DOI 10.1145/2939672.2939785). Bibliographic facts only; the paper itself was not read for this course.

How the overfitting curve works

1. Sum of trees

a fitted model holds every shorter model

2. Two row sets

the rows fitted on, and rows it was not

3. After k trees

iteration_range = (0, k)

4. Find the peak

the held-out curve stops, then falls

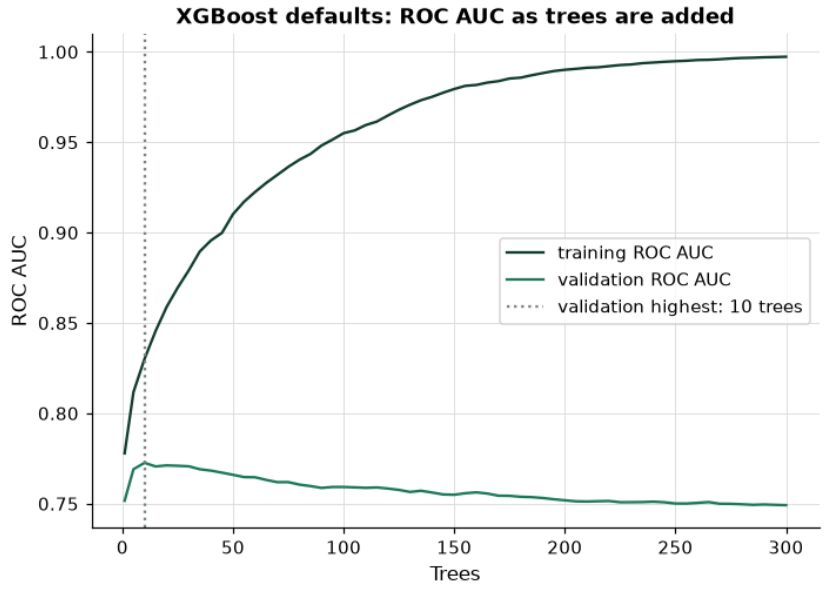
You control

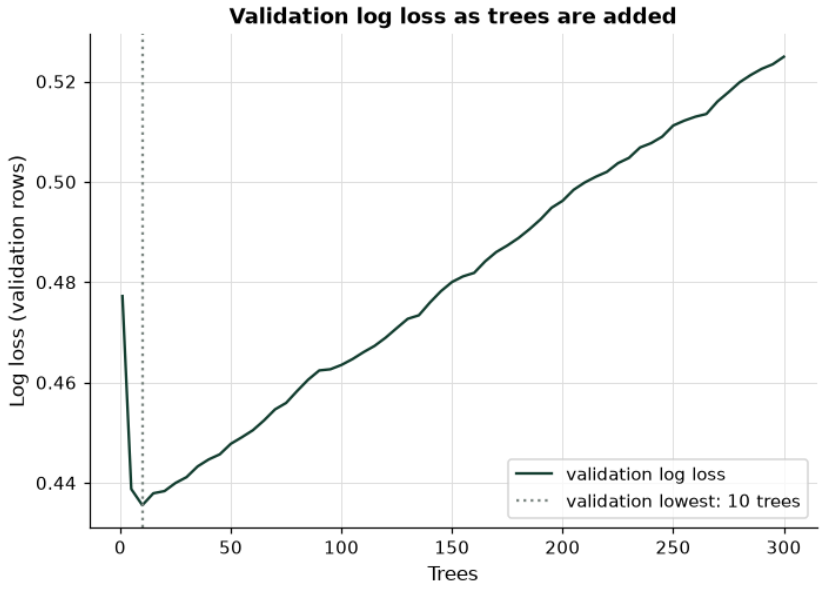
Which tree counts to score.

What comes out

Two curves and the number of trees where the held-out one peaks.

The defaults overfit





0.7726

validation, 10 trees

0.7492

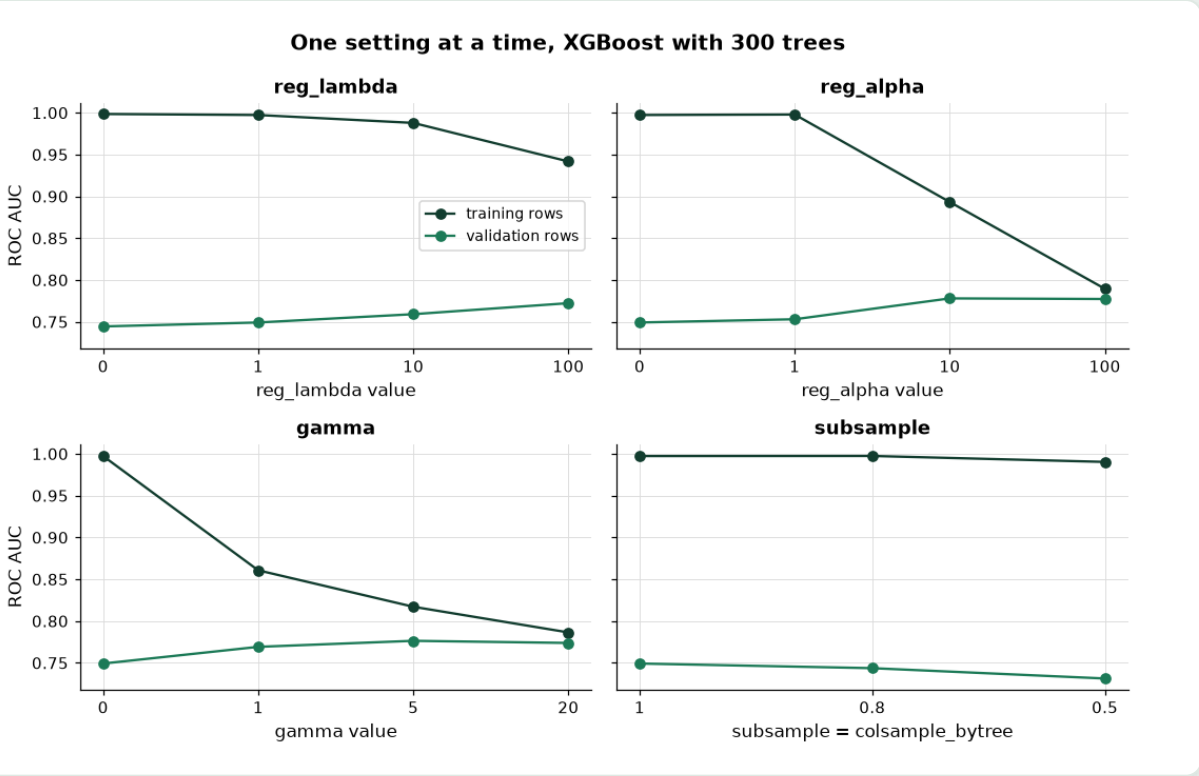
validation, 300 trees

0.9973

training, 300 trees

Validation log loss is lowest at 10 trees too. Described on the validation rows, chosen on nothing: the trees are chosen in 8.5.

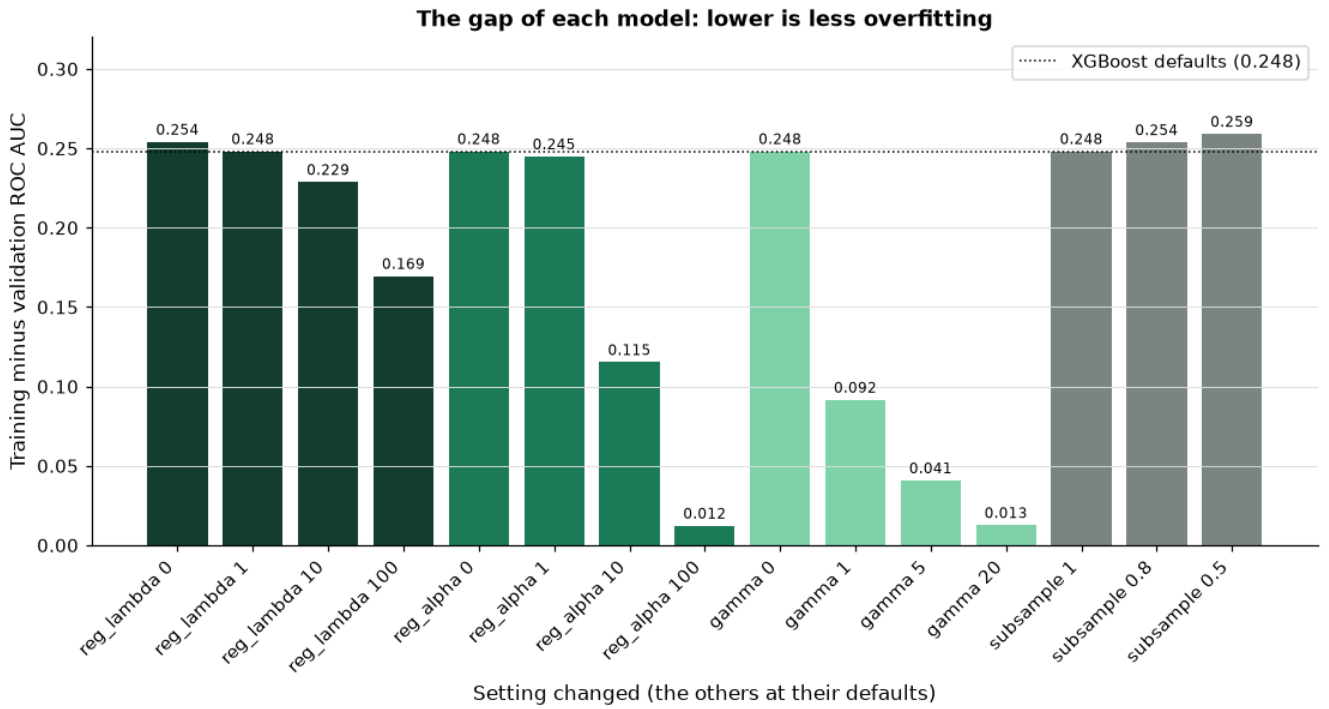
Each brake, one at a time



Setting	Training	Validation
defaults	0.9973	0.7492
reg_lambda 100	0.9417	0.7723
reg_alpha 10	0.8932	0.7780
gamma 5	0.8171	0.7763
sampling 0.8	0.9974	0.7436
sampling 0.5	0.9903	0.7313

Three brakes narrow the gap. Sampling alone does not, at learning rate 0.3 and depth 6. Nothing is chosen here.

The gap of each model



Lower is less overfitting. reg_alpha 100 and gamma 20 close the gap by pulling training ROC AUC down to 0.7895 and 0.7864.

How early stopping works

1. Split once

training rows into fit rows and stopping rows

2. Fit and score

the stopping rows after every tree

3. Wait

patience rounds without improvement

4. Keep the best

best_iteration; predict uses that many trees

You control

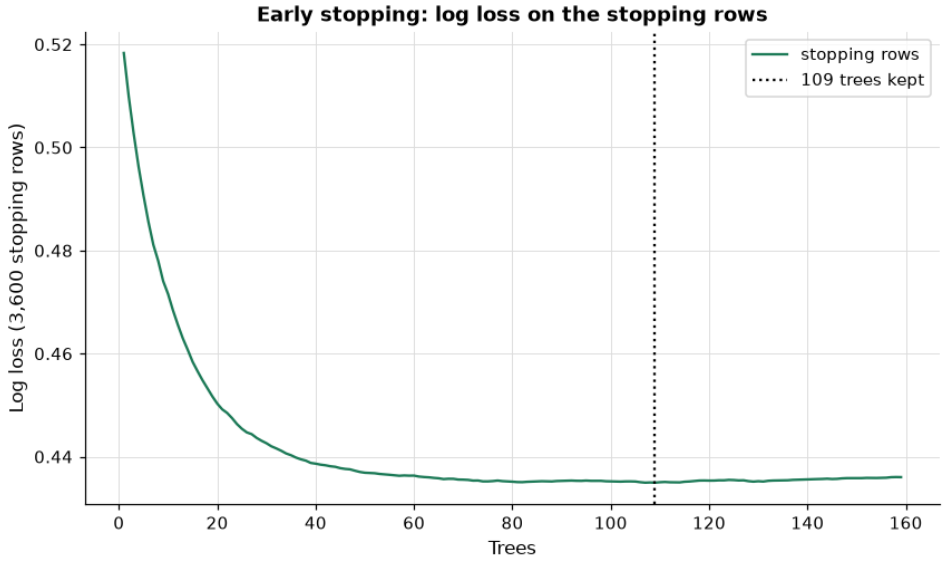
The patience, the metric, the most trees, and the size of the slice.

What comes out

best_iteration, and a model that scores with exactly that many trees. The stopping rows never are the validation rows.

Early stopping on rows carved from the training rows

14,400 fit rows	3,600 stopping	validation: report



109

trees kept

0.7788

validation ROC AUC

159 rounds run; best_iteration 108 (counted from 0). Stopping-row log loss 0.4350.

The rows that stop the trees choose the model, so they come from the training rows.

How scale_pos_weight works

1. Few defaults

about a fifth of the training accounts

2. Multiply

gradient and hessian of every default row

3. Usual value

the number of 0s over the number of 1s

4. Scores rise

above the default rate

You control

That one number.

What comes out

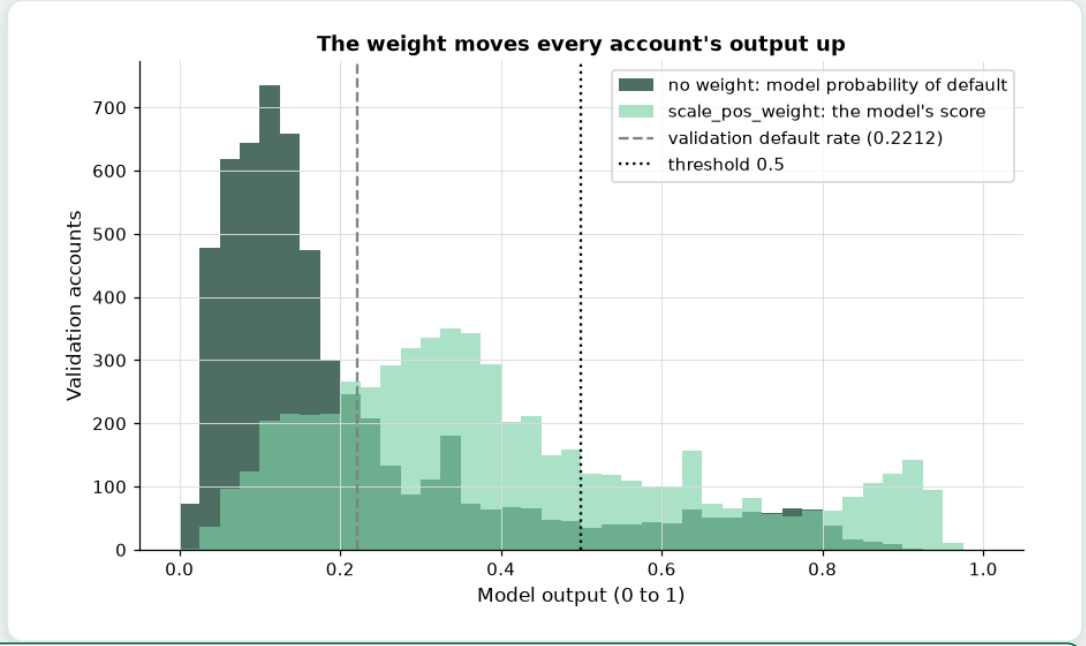
A model whose output is a score that ranks, never a probability.

The rare class: scale_pos_weight

```
scale_pos_weight = negatives / positives = 3.5203     starting probability with it: 0.5
```

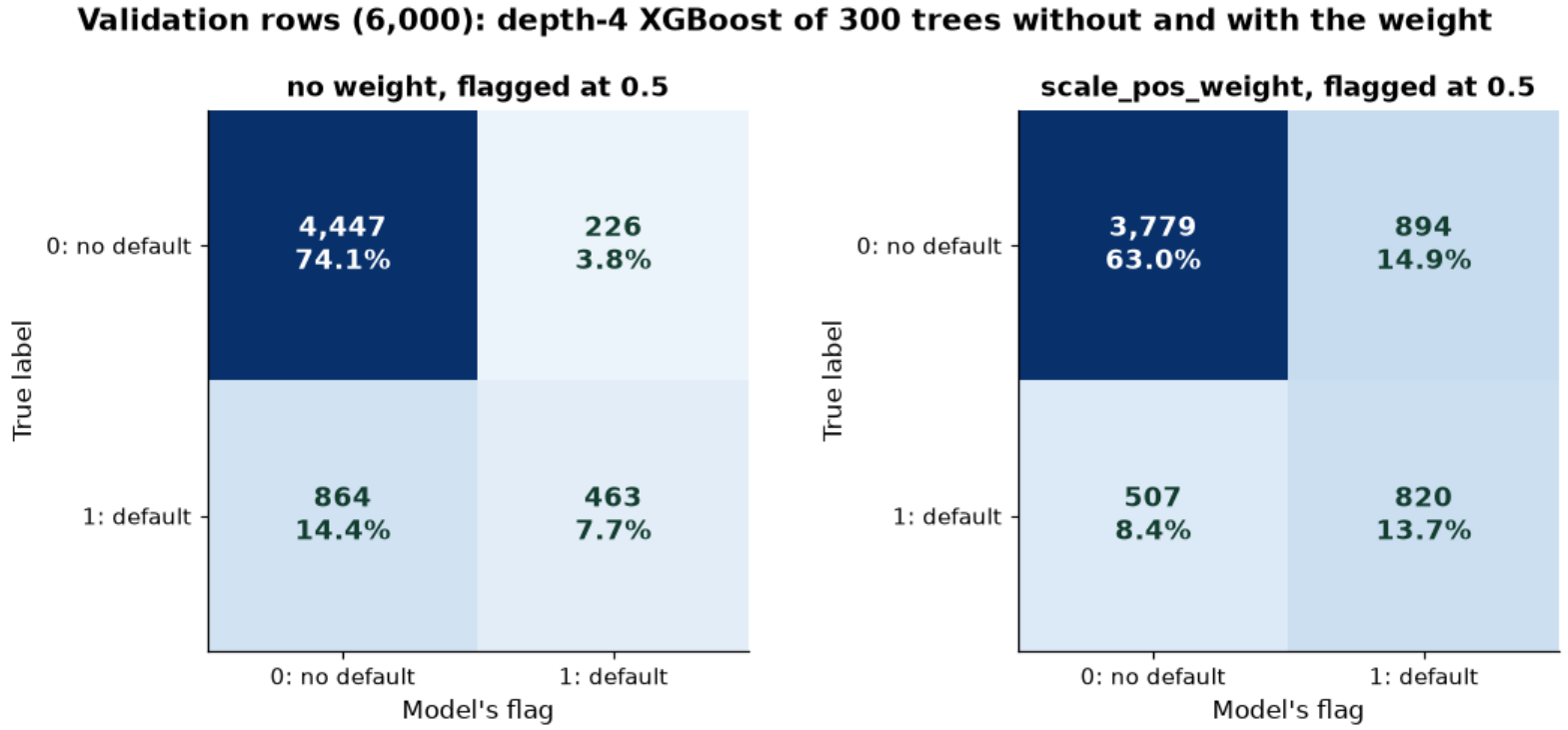
Validation rows	No weight	scale_pos_weight
Average output	0.2169	0.4085
ROC AUC	0.7794	0.7799
Average precision	0.5447	0.5478
Flagged at 0.5	11.5%	28.6%

The default rate is 0.2212. With the weight the average is 0.4085: the model's score, not a probability.



The weight moves every account's number, and where a threshold lands; the order barely moves.

Same ranking, different flags



11.5%

flagged, no weight

28.6%

flagged, with the weight

ROC AUC 0.7794 and 0.7799.

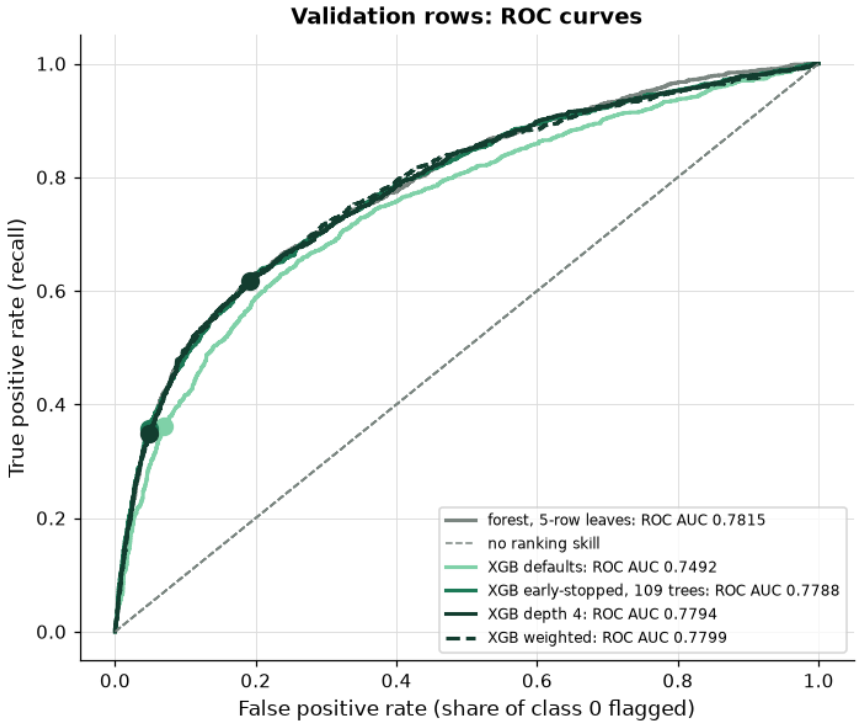
Read the right-hand column of each heatmap: the same ranking cut at a lower bar flags more of everything.

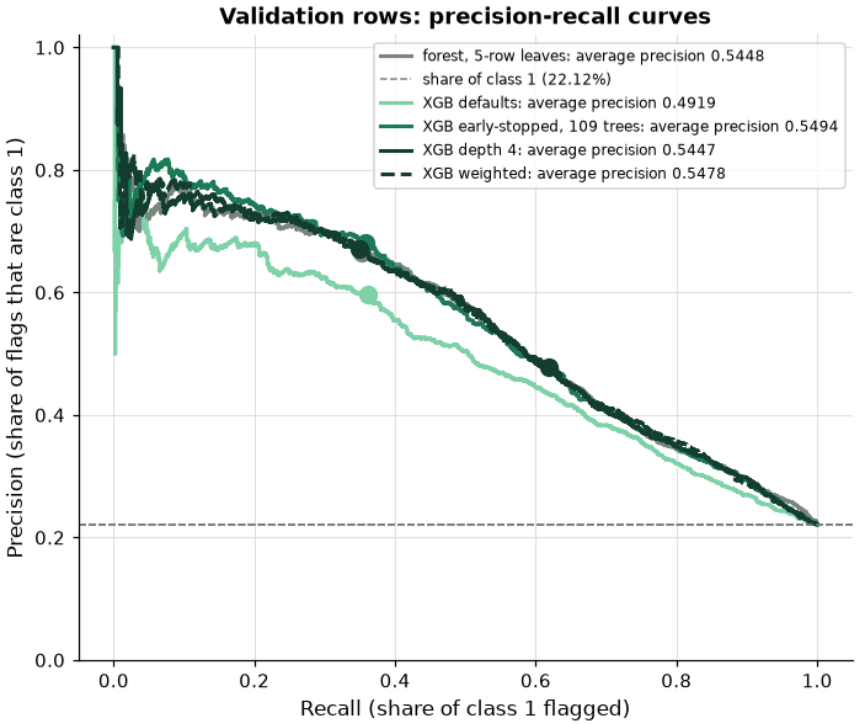
Beside Course 4 and week 1

Model	Validation ROC AUC	Average precision
day 3 forest, 300 trees, 5-row leaves	0.7815	0.5448
XGBoost 300 trees, scale_pos_weight	0.7799	0.5478
XGBoost 300 trees, depth 4	0.7794	0.5447
XGBoost early-stopped, 109 trees	0.7788	0.5494
Course 4 pruned tree	0.7627	0.4954
XGBoost defaults, 300 trees	0.7492	0.4919
Course 4 logistic regression	0.7190	0.4946

A few thousandths on one set of rows may be which rows were scored: day 10 measures that noise.

The ranking, as curves





Four models lie almost on one curve; the defaults are lowest. A dot is one threshold on the ranking.

How log loss works

1. The probability

the model gave what happened

2. Negative log

small if confident and right, large if wrong

3. Average

over all accounts

4. Lower is better

every tree today lowers it

You control

Nothing: the outcomes and the probabilities fix it.

What comes out

One number, computed by hand and checked in Excel.

Log loss, by hand

```
terms = -(y * np.log(p) + (1 - y) * np.log(1 - p));   log loss = terms.mean()
```

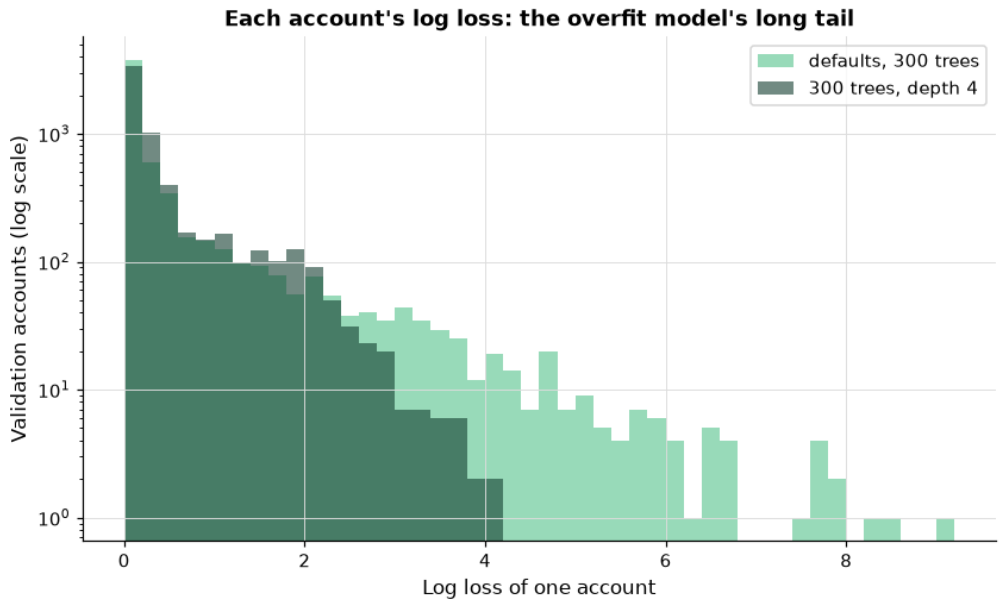
0.4323

300 trees, depth 4

0.5249

defaults, 300 trees

Validation rows	Defaults	Depth 4
Worst account	9.0298	4.1679
Median account	0.0995	0.1678
Accounts above 2	511	245



An overfit model is confidently wrong on a few accounts, and log loss charges a confident miss heavily.

Every setting of XGBClassifier

Setting	Constructor	Really used	Tune first?
learning_rate	None	0.3	tune first
n_estimators	None	100 trees	tune first
max_depth	None	6	tune first
min_child_weight	None	1	tune first
reg_lambda	None	1	tune first
reg_alpha	None	0	
gamma	None	0	
subsample	None	1	tune first
colsample_bytree	None	1	tune first
scale_pos_weight	None	1	

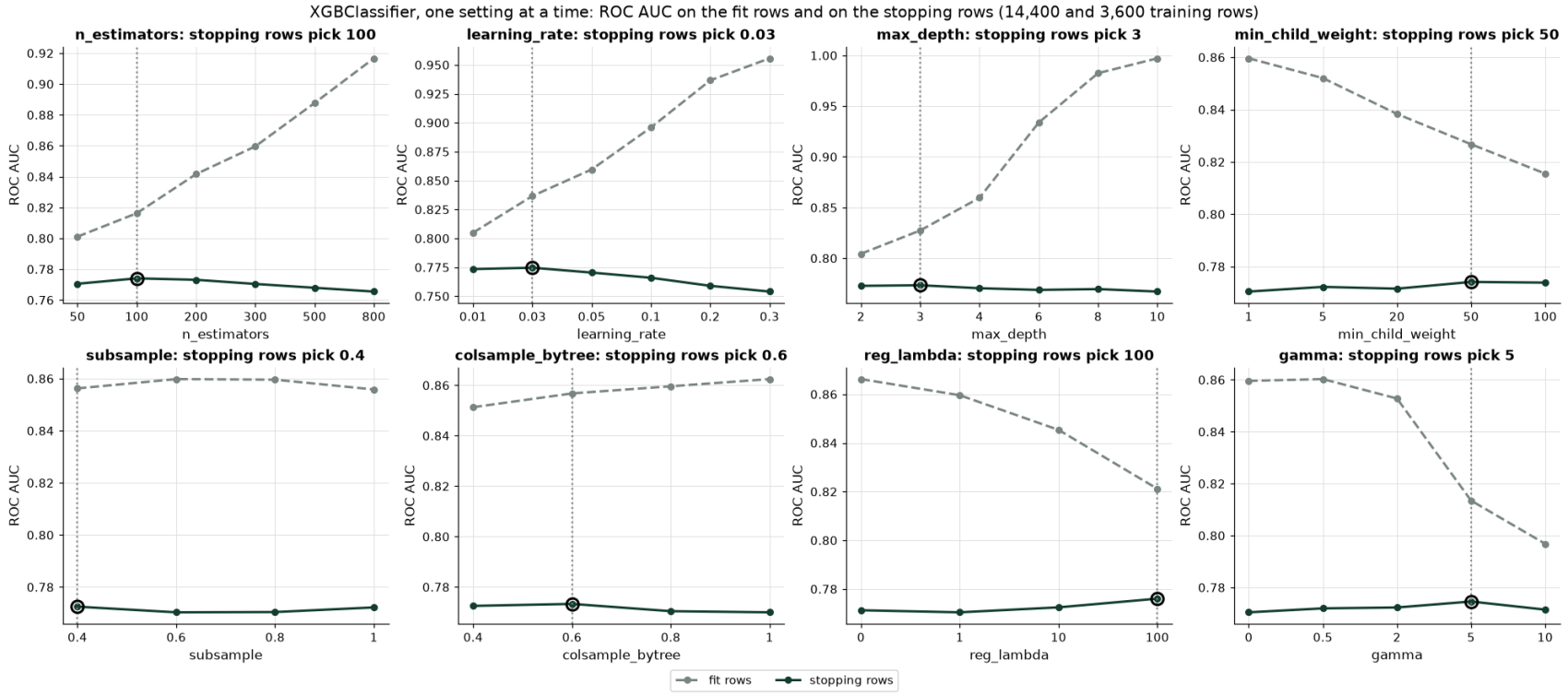
None means the library's default

Most constructor defaults are None. The second column is read from the saved config of a fitted model: eta 0.3, depth 6, lambda 1, every row and column, the histogram updater.

Tune first

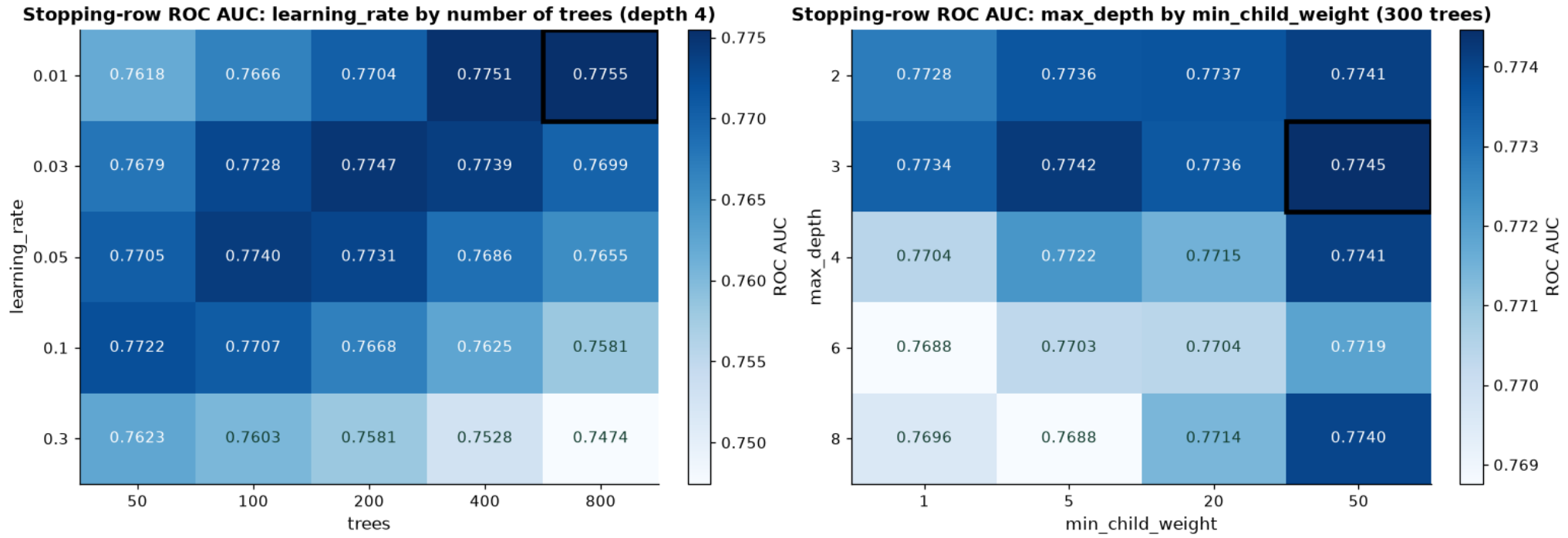
Learning rate with the number of trees, depth, min_child_weight, reg_lambda, and the two sampling shares. 40 settings in all.

One setting at a time, on the stopping rows



The fit rows' ROC AUC moves a lot; the stopping rows' moves a few thousandths. Settings change what is memorised far more than what holds on new rows.

Learning rate against trees, and tree shape



A smaller rate needs more trees and peaks later: they are one choice, and early stopping makes it.

Excel side by side

Excel side by side (tested in Excel)

`=AVERAGE(A2:A18001)`	starting probability 0.2212
`=$E$1-A2  and  =$E$1*(1-$E$1)`	gradient and hessian, filled down
`=SUMIFS(C2:C18001,B2:B18001,"<2")`	G left 905.00
`=-0.3*I4/(J4+H4)`	left leaf -0.097755
`=I4^2/(J4+H4)+K4^2/(L4+H4)-(I4+K4)^2/(J4+L4+H4)`	gain 2809.1
`=COUNTIF(A2:A18001,0)/COUNTIF(A2:A18001,1)`	scale_pos_weight 3.5203
`=-(A2*LN(B2)+(1-A2)*LN(1-B2)), then =AVERAGE(C2:C6001)`	log loss 0.4323

Excel has no XGBoost. These formulas reproduce one leaf, the weight, and log loss, each equal to Python within 1e-12.

Save the day: git diff --word-diff

1. Code

mlkit.py, ensemblekit, its tests

2. Log at 0.5

four XGBoost models, committed

3. Thresholds

out-of-fold, recall 0.70, training rows

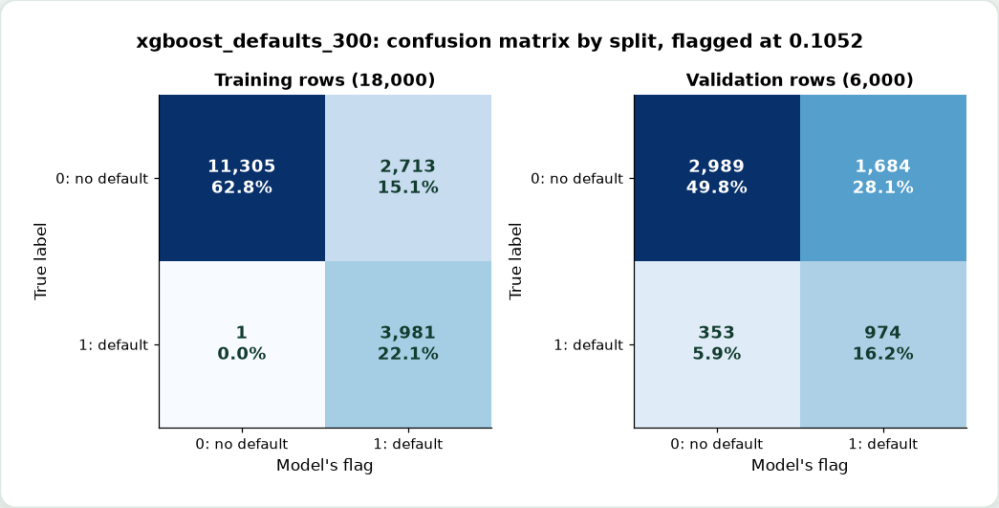
4. Word diff

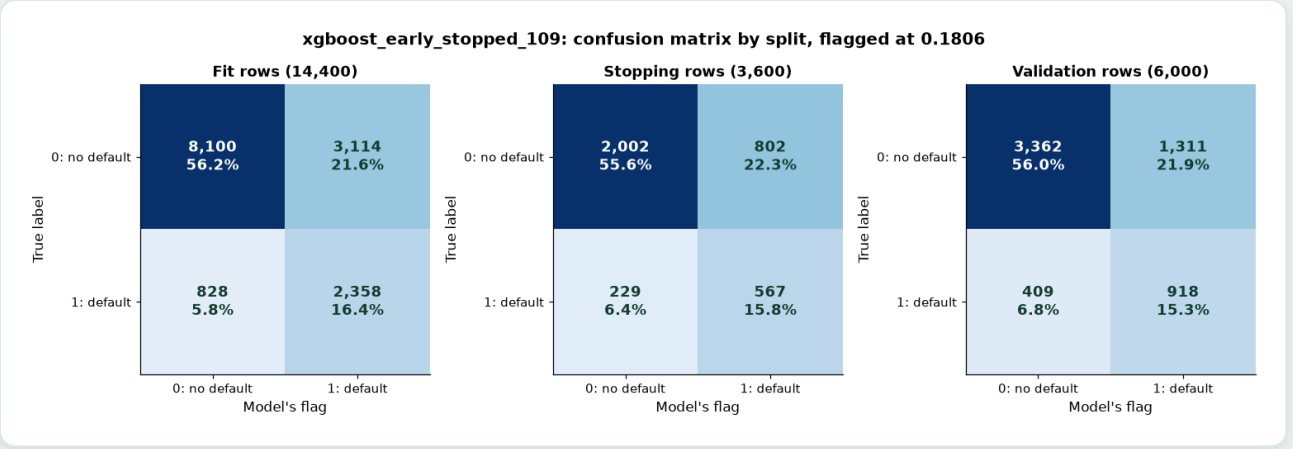
the change inside each line

```
xgboost_defaults_300,validation,0.7492,0.4919,[-0.5000,0.8048,0.5970,0.3617,0.4505-]{+0.1052,0.6605,0.3664,0.7340,0.4888+}
xgboost_early_stopped_109,validation,0.7788,0.5494,[-0.5000,0.8210,0.6815,0.3580,0.4694-]{+0.1806,0.7133,0.4118,0.6918,0.5163+}
```

Thresholds: 0.1812 unweighted, 0.4206 weighted. ROC AUC does not move with a threshold.

Each model by split, at its own threshold





Defaults: its own rows learned by heart, new rows not. Early-stopped: fit, stopping, and validation panels alike.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows choose, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: early stopping watches the validation rows, then reports ROC AUC on the same rows.

Code that runs is not the same as code that is right.

What you can do now

- An XGBoost leaf by hand: -lr x G / (H + lambda)
- gamma removes splits whose gain is below it
- n_jobs=1 on every XGBoost model
- Stop early on rows carved from the training rows
- scale_pos_weight gives the model's score; 32 tests pass

Week 2, Day 4 LightGBM, and stacking on out-of-fold outputs